Laborübung 2 – Internet of Things

Sensor Data Acquisition and MQTT Communication with ESP32

Berliner Hochschule für Technik

Submitted by

Mazen Younis
Mert Satici

Faculty Department VI – Computer Engineering
Berlin University of Applied Sciences and Technology

Submission Date: 20 May 2026



Studiere Zukunft

Supervisor
Prof. Dr.-Ing. Thomas Loewel Berliner Hochschule für Technik

1 Labor 2

1.1 Aufgabe: Sensorwerte mit dem ESP32 auslesen und per MQTT übertragen

In dieser Laborübung wurde ein BME280-Sensor mit dem ESP32 verbunden. Der Sensor misst Temperatur, Luftfeuchtigkeit und Luftdruck. Die ausgelesenen Werte werden anschließend über WLAN mit MQTT an einen MQTT-Broker gesendet.

Verwendete Komponenten

- ESP32-WROVER-DEV Board
- BME280 Sensor
- Breadboard
- Jumper-Kabel
- Visual Studio Code mit ESP-IDF Plugin

Schaltung

Der BME280-Sensor wurde über I2C mit dem ESP32 verbunden. Dafür wurden die Pins SDA und SCL verwendet.

BME280 Sensor	**ESP32**
VIN	3.3V
GND	GND
SDA	GPIO21
SCL	GPIO22

Tab. 1: Verbindung zwischen BME280-Sensor und ESP32

MQTT-Test

Vor der Integration des Sensors wurde zunächst geprüft, ob der ESP32 erfolgreich eine WLAN-Verbindung herstellen und Daten über MQTT veröffentlichen kann. Dafür wurde zunächst ein einfacher Zählerwert als Testnachricht gesendet.

```
1 Labor2: Connected to WiFi
2 Labor2: MQTT connected
3 Labor2: Published message: {"sensor_value": 0}
```

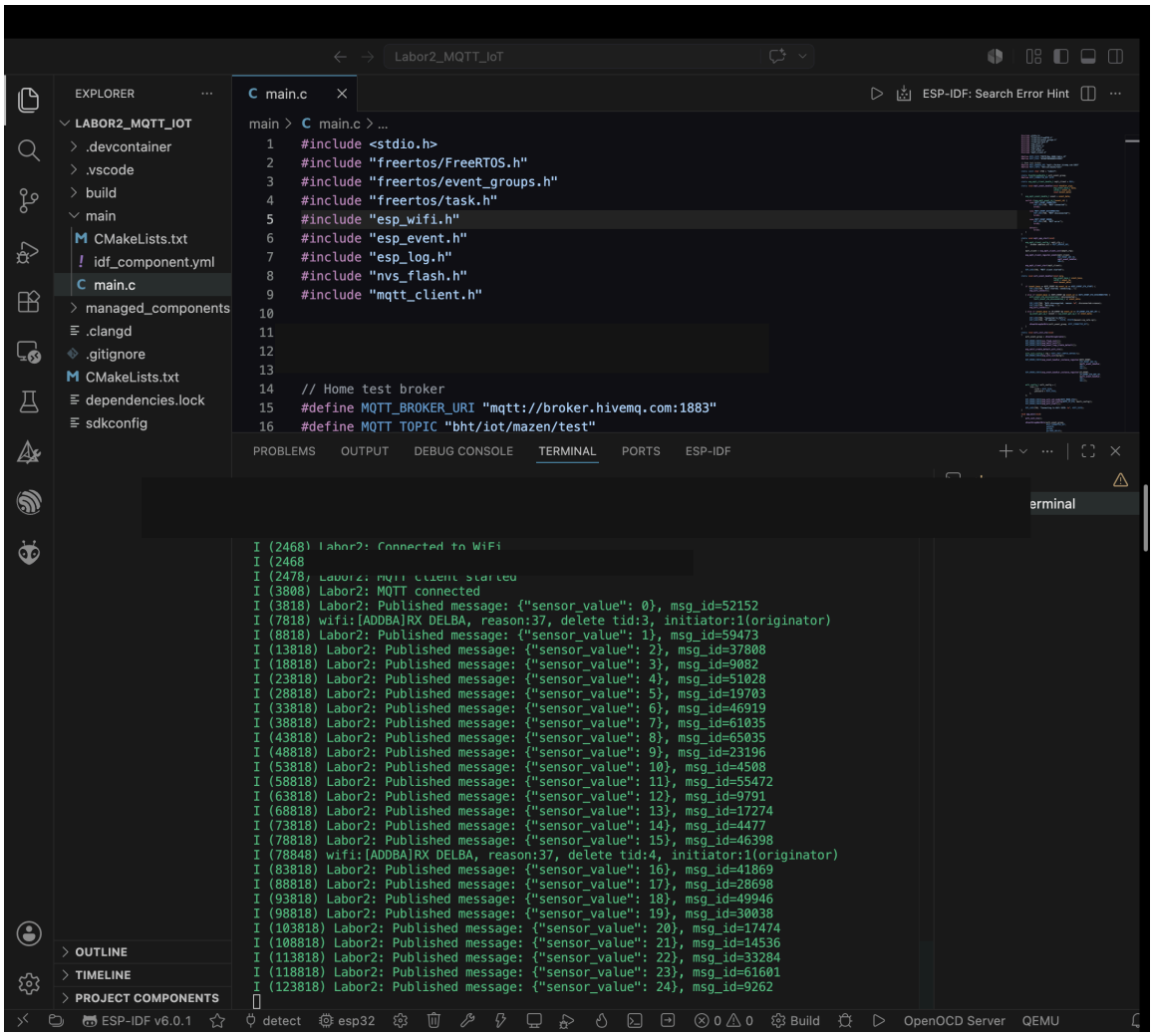


Fig. 1: MQTT-Testprogramm vor der Integration des Sensors

Der Test bestätigte, dass die WLAN-Verbindung sowie die MQTT-Kommunikation erfolgreich funktionieren.

I2C-Test

Zuerst wurde geprüft, ob der Sensor korrekt über I2C erkannt wird. Dazu wurde ein I2C-Scanner verwendet.

```
I2C_SCAN: Starting I2C scan...
I2C_SCAN: I2C device found at address: 0x76
I2C_SCAN: I2C scan finished
```

Der Sensor wurde erfolgreich unter der I2C-Adresse `0x76` erkannt.

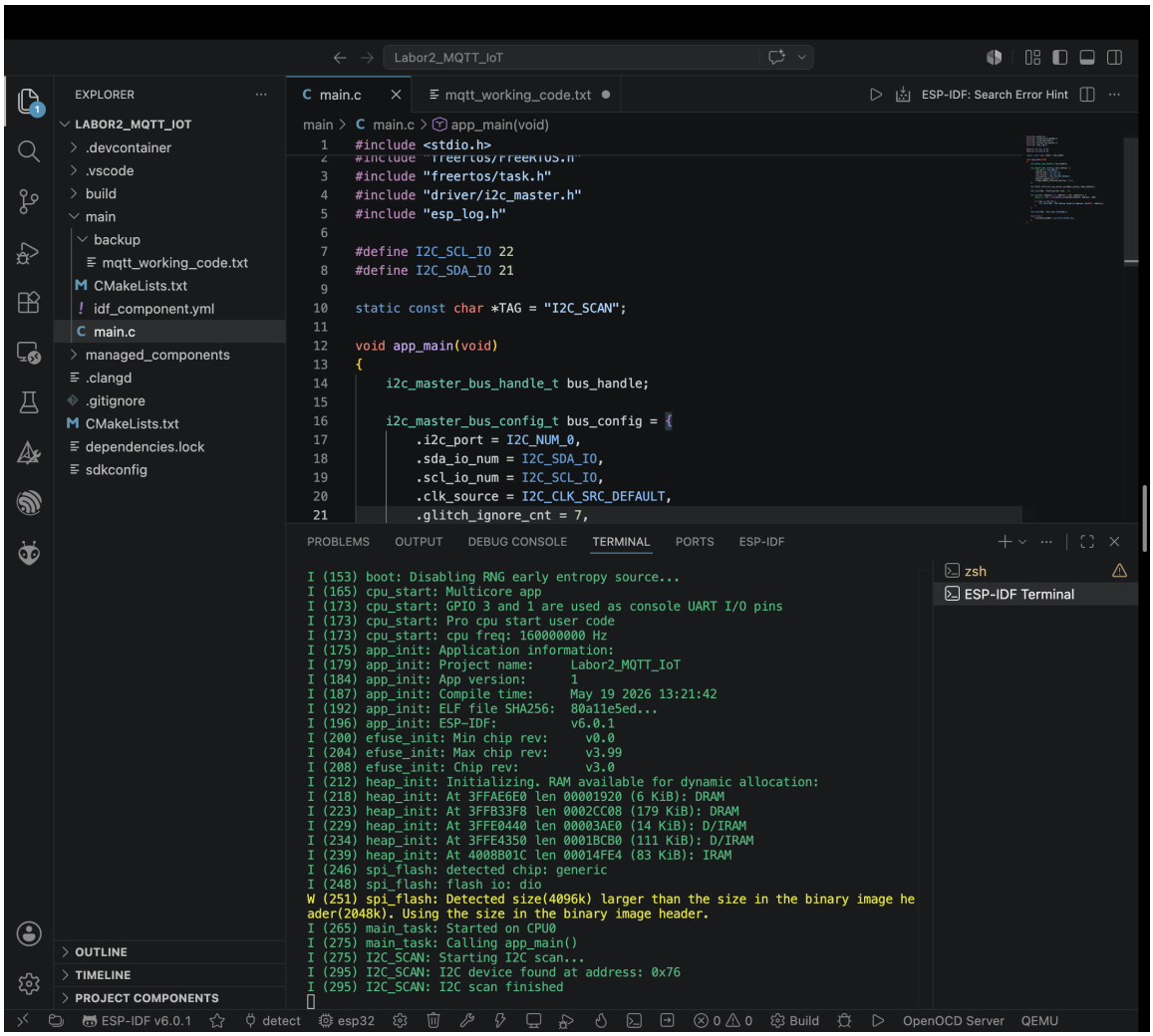


Fig. 2: Erfolgreicher I2C-Scan des BME280-Sensors

Sensorinitialisierung

Anschließend wurde der Chip des Sensors geprüft. Dabei wurde die Chip-ID `0x60` ausgelesen. Diese Chip-ID bestätigt, dass es sich um einen BME280-Sensor handelt.

```
BMP280_TEST: BMP280 found
BMP280_TEST: Chip ID: 0x60
```

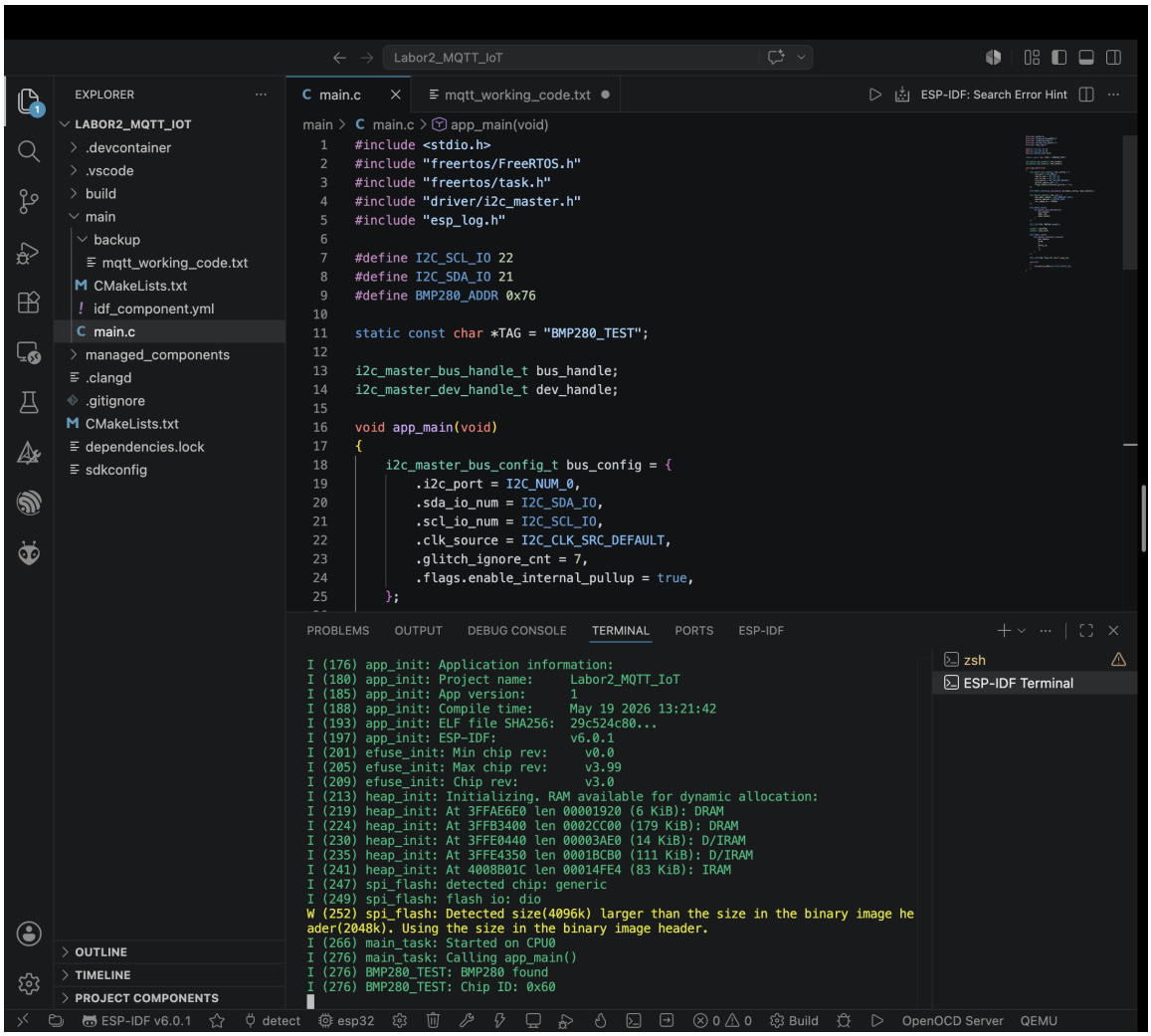


Fig. 3: Auslesen der Chip-ID des BME280-Sensors

Auslesen der Sensorwerte

Nach der erfolgreichen Initialisierung wurden Temperatur, Luftfeuchtigkeit und Luftdruck ausgelesen.

```
BME280_TEST: BME280 initialized successfully
BME280_TEST: Temperature: 25.73 C
BME280_TEST: Humidity: 37.84 %
BME280_TEST: Pressure: 1011.14 hPa
```

Die gemessenen Werte waren plausibel und wurden regelmäßig im Terminal ausgegeben.

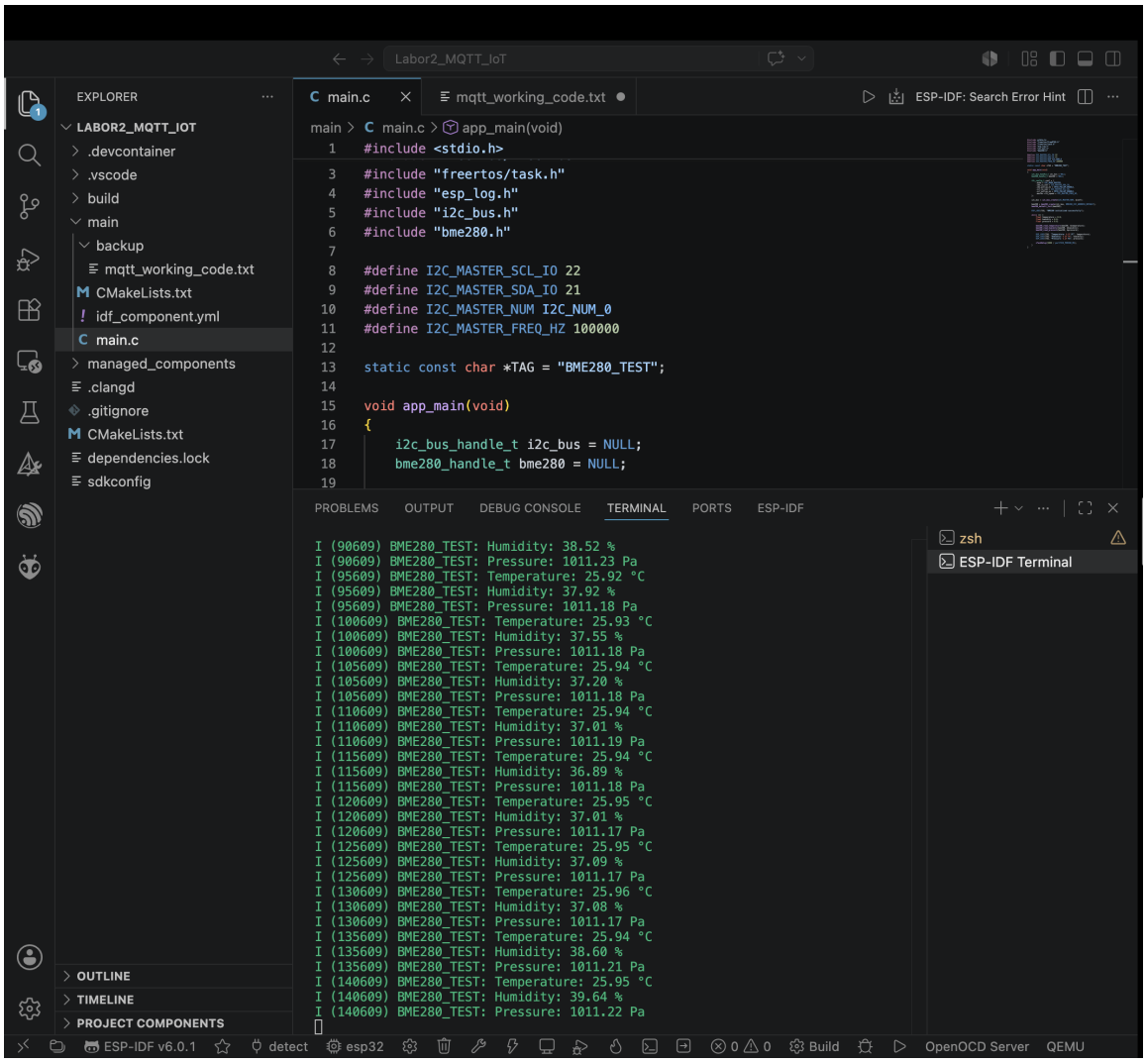


Fig. 4: Ausgabe der gemessenen BME280-Sensorwerte

WLAN-Verbindung

Für erste Funktionstests zu Hause wurde der ESP32 zunächst mit einem privaten WLAN verbunden. Im Labor muss später das vorgegebene WLAN aus der Aufgabenstellung verwendet werden:

`ESP32_MQTT_Broker`

Die Verbindung wurde erfolgreich hergestellt und der ESP32 erhielt eine IP-Adresse.

```
Labor2: Connected to WiFi
Labor2: IP address: [PRIVATE_IP]
```

MQTT-Kommunikation

Für den Test wurde zunächst ein öffentlicher MQTT-Broker verwendet:

`mqtt://broker.hivemq.com:1883`

Als Topic wurde verwendet:

`bht/iot/mazen/test`

Nach der erfolgreichen Verbindung mit dem MQTT-Broker wurden die Sensorwerte als JSON-Nachricht veröffentlicht.

```
Labor2: MQTT connected
Labor2: Published sensor data:
{"temperature": 24.33, "humidity": 36.53, "pressure": 1011.09}
```

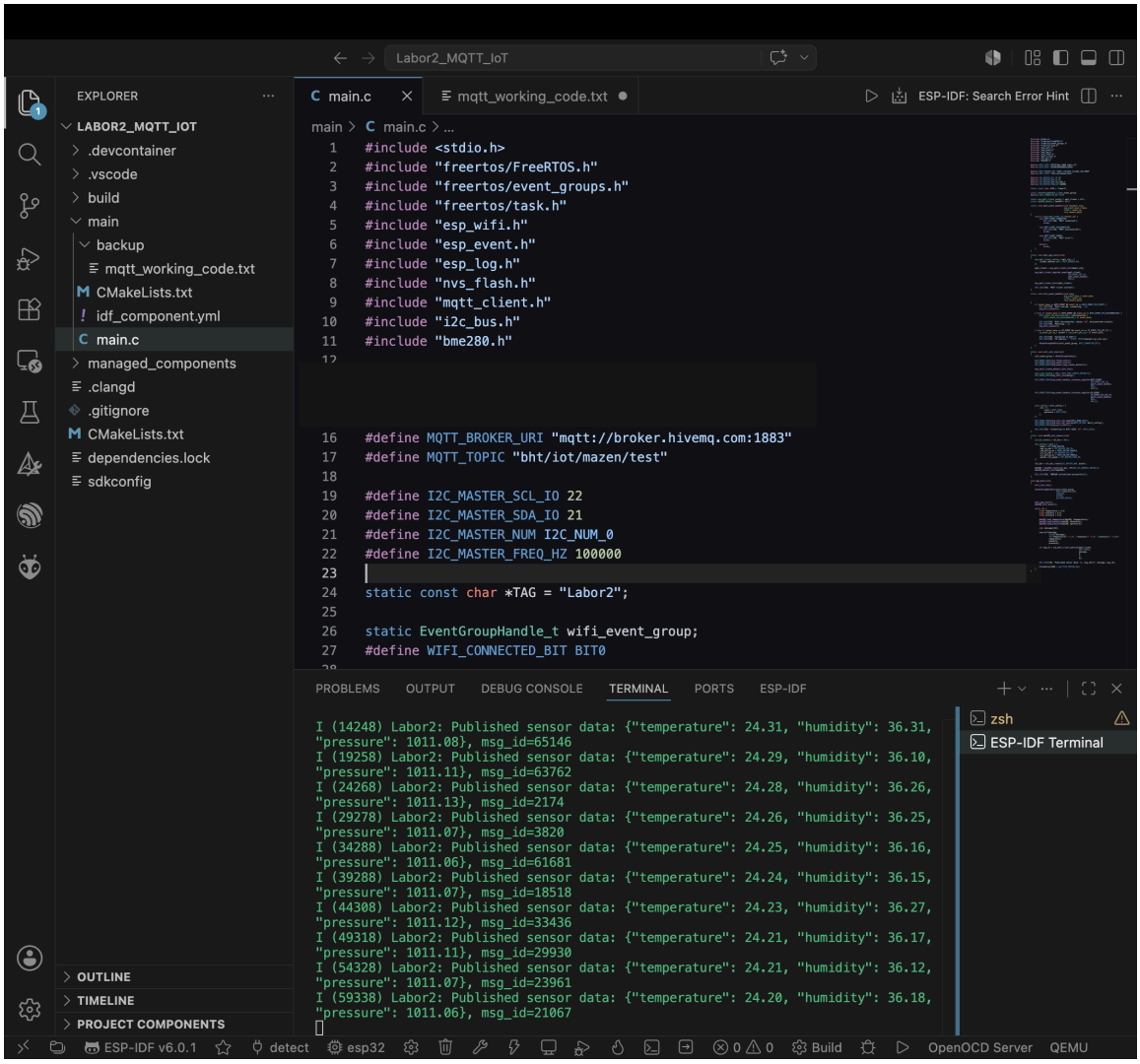


Fig. 5: Veröffentlichung der BME280-Sensorwerte über MQTT

Quellcode

Der folgende Quellcode verbindet den ESP32 mit dem WLAN, initialisiert den BME280-Sensor und sendet die gemessenen Werte regelmäßig über MQTT.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
```

```
3  #include "freertos/event_groups.h"
4  #include "freertos/task.h"
5  #include "esp_wifi.h"
6  #include "esp_event.h"
7  #include "esp_log.h"
8  #include "nvs_flash.h"
9  #include "mqtt_client.h"
10 #include "i2c_bus.h"
11 #include "bme280.h"
12
13 #define WIFI_SSID "PRIVATE_SSID"
14 #define WIFI_PASS "********"
15
16 #define MQTT_BROKER_URI "mqtt://broker.hivemq.com:1883"
17 #define MQTT_TOPIC "bht/iot/mazen/test"
18
19 #define I2C_MASTER_SCL_IO 22
20 #define I2C_MASTER_SDA_IO 21
21 #define I2C_MASTER_NUM I2C_NUM_0
22 #define I2C_MASTER_FREQ_HZ 100000
23
24 static const char *TAG = "Labor2";
25
26 static EventGroupHandle_t wifi_event_group;
27 #define WIFI_CONNECTED_BIT BIT0
28
29 static esp_mqtt_client_handle_t mqtt_client = NULL;
30 static bme280_handle_t bme280 = NULL;
31
32 /* MQTT event handling, WiFi setup and BME280 initialization
33    were implemented as separate functions. */
34
35 void app_main(void)
36 {
37     wifi_init_sta();
38
39     xEventGroupWaitBits(wifi_event_group,
40                         WIFI_CONNECTED_BIT,
41                         pdFALSE,
42                         pdTRUE,
43                         portMAX_DELAY);
44
45     mqtt_app_start();
46     bme280_init_sensor();
47
48     vTaskDelay(1000 / portTICK_PERIOD_MS);
49
50     while (1) {
```

```
        float temperature = 0.0;
        float humidity = 0.0;
        float pressure = 0.0;

        bme280_read_temperature(bme280, &temperature);
        bme280_read_humidity(bme280, &humidity);
        bme280_read_pressure(bme280, &pressure);

        char message[128];

        snprintf(message,
                sizeof(message),
                "{\"temperature\": %.2f, \"humidity\": %.2f, \"pressure\": %.2f}",
                temperature,
                humidity,
                pressure);

        esp_mqtt_client_publish(mqtt_client,
                                MQTT_TOPIC,
                                message,
                                0,
                                1,
                                0);

        ESP_LOGI(TAG, "Published sensor data: %s", message);

        vTaskDelay(5000 / portTICK_PERIOD_MS);
    }
}
```

Listing 1: ESP32 Programm zur Übertragung der BME280-Sensorwerte per MQTT

Anpassung für das Labor

Für die finale Ausführung im Labor müssen die WLAN- und MQTT-Daten auf die Vorgaben aus der Aufgabenstellung angepasst werden:

```
#define WIFI_SSID "ESP32_MQTT_Broker"
#define WIFI_PASS "REDACTED"

#define MQTT_BROKER_URI "mqtt://LOCAL_BROKER:1883"
```

Das MQTT-Topic kann in der Übung gemeinsam festgelegt werden.

Ergebnis

Die Laborübung wurde erfolgreich vorbereitet und getestet. Der ESP32 liest die Messwerte des BME280-Sensors aus und sendet Temperatur, Luftfeuchtigkeit und Luftdruck regelmäßig als JSON-Nachricht über MQTT an einen Broker.